

Ứng dụng của thuật toán Euclid

Kim Bân

Trường Trung học Cao cấp Thường Châu, Giang Tô

Tháng 1 năm 2009

Nguồn gốc: Applications of the Euclidean Algorithm

I0I-China-Candidate-Team-Papers/2009/Jin-Bin/euclidean-algorithm-applications.pdf

Abstract

Thuật toán chia Euclid để tìm ước chung lớn nhất của hai số là một trong những thuật toán sớm nhất được các nhà toán học nghiên cứu, đồng thời có quan hệ chặt chẽ với các nội dung của lý thuyết số như liên phân số và phương trình Diophantine. Bài viết bắt đầu từ thuật toán Euclid cơ bản, đề cập cách giải một số bài toán số học, rồi từ tư tưởng của nó nghiên cứu và giải quyết một vài bài toán thoát nhìn không liên quan tới lý thuyết số.

Từ khóa: thuật toán Euclid, chia lấy dư, liên phân số, phương trình Pell.

Contents

1	Thuật toán Euclid	2
1.1	Ước chung lớn nhất	2
1.1.1	Chia lấy dư liên tiếp	2
1.1.2	Phân tích độ phức tạp	2
1.2	Thuật toán Euclid mở rộng	3
2	Những thuật toán Euclid “khác thường”	3
2.1	Đa thức	4
2.2	Định thức ma trận theo modulo	4
2.3	Thuật toán Euclid hai chiều	5
2.4	Tiểu kết	7
3	Liên phân số và phương trình Pell	7
3.1	Liên phân số	7
3.2	Phương trình Pell	8
4	Một vài ứng dụng khác	11
4.1	Phân số có mẫu nhỏ nhất giữa hai phân số	11
4.2	Số điểm lưới trong đa giác đơn có đỉnh hữu tỉ	11

1 Thuật toán Euclid

Phần này chỉ giới thiệu ngắn gọn một số kiến thức cơ sở về thuật toán Euclid. Nếu bạn đã đủ quen thuộc với nó, có thể bỏ qua phần này.

1.1 Ước chung lớn nhất

Bài toán ước chung lớn nhất là một trong những bài toán thuật toán được nghiên cứu sớm nhất, xuất hiện trong *Cơ sở* của Euclid khoảng năm 300 trước Công nguyên. Nó cũng là nền tảng cho rất nhiều nội dung lý thuyết số trong thi tin học, chẳng hạn phương trình đồng dư tuyến tính và hệ phương trình đồng dư tuyến tính. Thuật toán Euclid, tức “phép chia Euclid” mà phần lớn thí sinh biết tới, có cốt lõi là liên tục làm nhỏ quy mô hai số, cuối cùng biến bài toán về quy mô có thể phán đoán trực tiếp trong thời gian logarit, nếu giả sử các phép toán số học cơ bản đều có chi phí $\mathcal{O}(1)$.

1.1.1 Chia lấy dư liên tiếp

Vì đây là thuật toán cơ bản nhất của lý thuyết số, ta trực tiếp đưa ra mã giả.

Algorithm 1 Euclidean Algorithm

Input: $x \geq 0$ and $y \geq 0$

Output: the greatest common divisor of x and y

```

1: while  $y \neq 0$  do
2:    $t \leftarrow x \bmod y$ 
3:    $x \leftarrow y$ 
4:    $y \leftarrow t$ 
5: end while
6: return  $x$ 

```

Sau đây là chứng minh ngắn gọn về tính đúng đắn. Với hai số bất kỳ x và y , đặt $x = y \times a + b$. Với mọi $d \mid x$ và $d \mid y$, ta có $x'd = y'd \times a + b$, nên $b = (x' - y' \times a) \times d$. Do đó $d \mid (x \bmod y)$, chứng tỏ kết quả cuối cùng *ans* chắc chắn là ước chung của x và y . Tương tự, với mọi ước chung d của x và y , đều có $d \mid \text{ans}$. Vì vậy *ans* là ước chung lớn nhất.

1.1.2 Phân tích độ phức tạp

Ở đây chỉ phân tích sơ lược để thấy độ phức tạp thời gian, bỏ qua chi phí của phép chia, là $\mathcal{O}(\log n)$. Với hai số bất kỳ a, b thỏa mãn $a > b$, xét hiệu quả của lệnh

$$a \leftarrow a \bmod b.$$

- Nếu $b \leq a/2$, thì $a \bmod b < b \leq a/2$.
- Nếu $b > a/2$, thì $a \bmod b \leq a - b < a - a/2 = a/2$.

Có thể thấy mỗi thao tác như vậy đều làm a giảm còn nhiều nhất một nửa, nên độ phức tạp cuối cùng là $\mathcal{O}(\log n)$. Trong lập luận này ta tạm bỏ qua lỗi có thể xảy ra khi lấy modulo cho 0.

1.2 Thuật toán Euclid mở rộng

Thuật toán Euclid mở rộng dựa trên thuật toán Euclid để giải một lớp phương trình Diophantine tuyến tính đặc biệt:

$$ax + by = \gcd(a, b).$$

Khi $\gcd(a, b) = 1$, tức a, b nguyên tố cùng nhau, nghiệm của phương trình này thực chất tương ứng với nghịch đảo của a theo modulo b . Vì vậy thuật toán này trở thành cơ sở để giải phương trình đồng dư tuyến tính và hệ phương trình đồng dư tuyến tính, rồi được dùng rất rộng rãi trong thi tin học.

Trước hết vẫn đưa ra mã giả.

Algorithm 2 Extended Euclidean Algorithm

Input: a, b

Output: a solution to $ax + by = \gcd(a, b)$

```
1: function extended_gcd(a, b)
2:   if b = 0 then
3:     return (1, 0)
4:   else
5:     (x, y) <- extended_gcd(b, a mod b)
6:     return (y, x - (a div b) * y)
7:   end if
```

Phân tích ngắn gọn như sau. Đây là một phiên bản sửa đổi của phép chia Euclid, nên vẫn tính được $\gcd(a, b)$. Xét dòng 3: khi tình huống hiện tại là $a = \gcd(a, b)$ và $b = 0$, rõ ràng $x = 1, y = 0$ là một nghiệm. Vì thế ở dòng 5 ta có thể giả sử

$$xb + y(a \bmod b) = \gcd(a, b).$$

Khai triển về trái:

$$\begin{aligned}\gcd(a, b) &= xb + y(a - (a \operatorname{div} b)b) \\ &= xb + ya - y(a \operatorname{div} b)b \\ &= ya + (x - (a \operatorname{div} b)y)b.\end{aligned}$$

Do đó sau khi thuật toán tính xuôi ra $\gcd(a, b)$, nó tính được nghiệm trong quá trình trả về. Độ phức tạp hiển nhiên giống thuật toán ban đầu.

Thoạt nhìn thuật toán Euclid dường như không có gì thần kỳ. Nhưng khi nhận ra bản chất “từng bước thu nhỏ quy mô” của nó, tư tưởng này thường đem lại một cảm giác rất mới khi giải một số bài toán, kể cả các bài toán “máy tính toán học”.

2 Những thuật toán Euclid “khác thường”

Dễ hình dung rằng mọi cặp “số” có thao tác lấy modulo và phù hợp với một thứ tự bộ phận đều có thể dùng thuật toán Euclid. Nói chính xác hơn, đây là các phần tử trong một vành Euclid; một ví dụ là đa thức dưới đây.

Dường như ta còn có thể làm nhiều hơn: phóng đại thao tác modulo thành biến đổi sơ cấp của ma trận, thậm chí thành thao tác trên vector hai chiều.

2.1 Đa thức

Ước chung lớn nhất của đa thức không được dùng nhiều trong thi lập trình. Việc đặt nó ở đầu phần này chủ yếu có ý nghĩa khái niệm.

Ta đưa ra một định nghĩa không thật chính quy cho phép “chia hết” đa thức: tìm một đa thức mới sao cho bậc của “số dư” nhỏ hơn bậc của “số chia”. Từ đó có thể định nghĩa thao tác “lấy modulo” và quá trình Euclid phía sau. Dưới đây là một ví dụ:

$$x^4 + 8x^3 + 12x^2 + 17x + 6, \quad (1)$$

$$x^4 - 4x^3 + 4x^2 - 3x + 14, \quad (2)$$

$$x^3 + \frac{2}{3}x^2 + \frac{5}{3}x - \frac{2}{3}, \quad (3)$$

$$x^2 + x + 2, \quad (4)$$

$$0. \quad (5)$$

Từ đó có thể nhận được ước chung lớn nhất của (1) và (2) là (4). Ước chung lớn nhất của hai đa thức biểu thị các nghiệm chung của hai phương trình đa thức.

2.2 Định thức ma trận theo modulo

Nguồn bài. Một bài toán kinh điển; một ví dụ là SPOJ Find The Determinant III. Một ví dụ khác là Timus 1627: Join.

Đề bài. Cho một ma trận nguyên A kích thước $n \times n$. Hãy tính $\det(A) \bmod M$, trong đó M là số nguyên dương.

Phân tích và lời giải. Bài toán này được dùng ở nhiều nơi. Khi dùng định thức để tính số cây khung, đa số trường hợp chỉ cần đáp án theo modulo một số. Nếu modulo là số nguyên tố hoặc một số square-free, ta có thể xử lý bằng nghịch đảo modulo, hoặc dùng thêm định lý số dư Trung Hoa. Nhưng với modulo tổng quát, phép khử Gauss thông thường gặp nhiều vấn đề.

Lời giải bài toán này hiển nhiên vẫn dựa trên khử Gauss, nên độc giả chưa quen nên dừng lại ôn tập trước. Trước hết có hai cách rất trực tiếp:

1. Một cách hiển nhiên là viết một lớp phân số độ chính xác cao, rồi có thể làm tiếp. Nhưng vì các phép toán độ chính xác cao không hoàn thành trong $\mathcal{O}(1)$, hiệu quả cuối cùng không tốt, độ phức tạp khoảng $\mathcal{O}(n^4)$.
2. Ý tưởng ban đầu của người ra đề: trước hết chuyển bài toán về modulo một lũy thừa của số nguyên tố, chẳng hạn a^p , rồi cuối cùng hợp nhất bằng định lý số dư Trung Hoa. Khi cài đặt cụ thể, nếu các phần tử có thể chọn làm pivot hiện tại đều là bội của a , trước tiên chia cả cột này cho a , sau đó tính định thức modulo a^{p-1} rồi nhân lại với a . Cách này khá phức tạp; độ phức tạp cuối cùng là $\mathcal{O}(n^3 \log C)$.

Hai cách trên đều có thể giải bài nếu cài đặt tốt, nhưng trông không thật đẹp. Theo kiến thức đại số, nếu đổi chỗ hai hàng bất kỳ của ma trận, định thức đổi dấu; nếu cộng hoặc trừ một bội của một hàng vào một hàng khác, định thức không đổi. Để nhận ra biến đổi sơ cấp của ma trận nguyên và thao tác “lấy modulo” có sự giống nhau đáng ngạc nhiên. Xét ma trận $A_{n,n}$ và các chỉ số i, j, k ; ta có thể dùng thao tác

$$A_i \leftarrow A_i - A_j \times (A_{i,k} \text{ div } A_{j,k})$$

để thực hiện

$$A_{i,k} \leftarrow A_{i,k} \bmod A_{j,k}$$

mà không làm thay đổi giá trị định thức.

Tới đây thuật toán trở nên trực tiếp. Vì đã có “modulo”, ta có thể trong $\mathcal{O}(n \log C)$ thời gian làm một trong hai số cùng cột thành 0, rồi phối hợp với đổi hàng để hoàn thành quá trình khử.

Algorithm 3 Determinant mod M

Input: $n \times n$ matrix A, M

Output: $\text{Det}(A) \bmod M$

```
1: for all element A[i,j] in A do
2:   A[i,j] <- A[i,j] mod M
3: end for
4: ans <- 1
5: for i = 1 to n do
6:   for j = i + 1 to n do
7:     while A[j,i] != 0 do
8:       t <- A[i,i] div A[j,i]
9:       A[i] <- A[i] - A[j] * t
10:      swap A[i] and A[j]
11:      ans <- -ans
12:     end while
13:   end for
14:   if A[i,i] = 0 then
15:     return 0
16:   end if
17:   ans <- ans * A[i,i]
18: end for
19: return ans
```

Điểm then chốt để nghĩ ra thuật toán này là nhìn thấu quan hệ giữa phép chia nguyên và biến đổi sơ cấp của ma trận, khéo léo tương ứng hóa quá trình khử với quá trình Euclid, từ đó giải bài trong $\mathcal{O}(n^3 \log C)$.

Đáng nhắc thêm rằng khi M là số lẻ, cách cài đặt nhị phân của thuật toán Euclid cũng có thể dùng cho bài này. Tư tưởng giống thuật toán vừa mô tả và hiệu quả cao hơn, dù độ phức tạp không đổi.

2.3 Thuật toán Euclid hai chiều

Nguồn bài. Bài do WiNGeR của SPb IFMO cung cấp, xuất phát từ Maloyaroslavets Summer Camp 2008.

Đề bài. Cho hai vector a, b . Tìm các số nguyên x, y , không đồng thời bằng 0, sao cho $|ax + by|$ nhỏ nhất.

Phân tích và lời giải. Trước hết xét một trường hợp rất đặc biệt: a và b cùng đường thẳng. Dễ thấy đáp án cuối cùng thực ra là $\gcd(|a|, |b|)$. Điều này gợi ý rằng bài toán có thể liên quan tới thuật toán Euclid.

Xét một tình huống cực đoan: nếu góc giữa a và b rất nhỏ, nhỏ đến mức có thể bỏ qua, thì đáp án về bản chất vẫn là $\gcd(|a|, |b|)$. Điều này cũng có thể cho thấy bài toán cần được giải bằng thuật toán Euclid.

Mô tả hình. Hình đầu trong bài gốc vẽ hai vector a và b gần như cùng hướng; hình tiếp theo vẽ lưới sinh bởi a và b , trong đó bài toán trở thành tìm điểm lưới khác O có khoảng cách tới O nhỏ nhất. Một hình thứ ba cho thấy khi góc giữa hai vector càng nhỏ thì lưới càng dẹt, bài toán càng “không tầm thường”.

Để tiện thảo luận, ta quy ước $a \cdot b \geq 0$, tức a và b cùng hướng, và $|a| \leq |b|$. Bây giờ xét một trường hợp đặc biệt khác: giả sử góc α giữa a và b lớn hơn $\pi/3$, tức $\cos \alpha < 1/2$.

Kết luận 1. Nếu góc giữa a, b lớn hơn $\pi/3$, thì đáp án cần tìm là $\min(|a|, |b|)$.

Chứng minh. Đặt $p = |a|, q = |b|$, không mất tính tổng quát giả sử $p < q$. Khi đó

$$\begin{aligned} |ax + by| &= \sqrt{(px)^2 + (qy)^2 - 2pxqy \cos \alpha} \\ &\geq \sqrt{|px|^2 + |qy|^2 - 2|px||qy| \cos \alpha} \\ &\geq \sqrt{|px|^2 + |qy|^2 - |px||qy|} \\ &\geq \sqrt{(|px| - |qy|)^2 + |px||qy|}. \end{aligned}$$

Xét các trường hợp:

- Nếu $x = 0$, thì $y \neq 0$, nên $(|px| - |qy|)^2 = |qy|^2 \geq q^2 \geq p^2$.
- Nếu $y = 0$, thì $x \neq 0$, nên $(|px| - |qy|)^2 = |px|^2 \geq p^2$.
- Nếu cả hai đều khác 0, thì $|px||qy| \geq |p||q| \geq p^2$.

Vì vậy mục tiêu hiện tại là liên tục biến đổi hai cạnh để tăng góc giữa chúng.

Kết luận 2. Đáp án ứng với (a, b) giống đáp án ứng với $(a, b + ka)$, trong đó k là số nguyên.

Chứng minh. Giả sử $|ax + by|$ là đáp án cần tìm. Khi đó

$$|a(x - ky) + (b + ka)y|$$

cũng là giá trị ấy, tức nghiệm (x, y) tương ứng với nghiệm $(x - ky, y)$. Nếu $(x, y) \neq (0, 0)$ thì $(x - ky, y) \neq (0, 0)$. Chiều ngược lại chứng minh tương tự.

Điều này gợi ý hướng biến đổi, nhưng vẫn còn nhiều chi tiết cần xét: chọn k như thế nào, và làm sao đảm bảo kết thúc sau hữu hạn bước?

Ta nhắc lại quy ước $|a| \leq |b|$. Hình trong bài gốc có các điểm O, A, B, C, D, E như sau: $\overrightarrow{OA} = a$, $\overrightarrow{OB} = b$, $BE \perp OA$, $\overrightarrow{OC} = k\overrightarrow{OA}$, $\overrightarrow{OD} = (k + 1)\overrightarrow{OA}$, và E nằm trên đoạn CD .

Nếu lúc này $|OA| > |OE|$, thì dễ chứng minh $\angle OAB > \angle AOB$; có thể lấy điểm đối xứng A' của A qua E để thấy điều này. Ngược lại, nếu $|CE| < |DE|$ thì thay (OA, OB) bằng (CB, OA) ; nếu không thì thay bằng (DB, OA) .

Kết luận 3.

$$\max(\angle BCE, \angle BDE) > \angle AOB.$$

Chứng minh. Có thể chứng minh trực tiếp

$$\angle DCB = \angle AOB + \angle CBO > \angle AOB.$$

Tổng hợp lại, phép biến đổi này luôn làm góc giữa hai vector tăng lên.

Vì vậy một thuật toán dựa trên Euclid đã hình thành: mỗi lần dùng hàm bậc hai để tính giá trị k ở trên, rồi thực hiện biến đổi, cho đến khi góc vượt quá $\pi/3$. Do hạn chế kiến thức, tác giả không đưa ra được độ phức tạp chính xác của thuật toán này; theo kết quả thực nghiệm, về cơ bản nó kết thúc sau vài bước, có lẽ ở cấp $\mathcal{O}(\log C)$.

2.4 Tiểu kết

Bản chất của thuật toán Euclid là dùng một phép toán đặc định để làm quy mô nhỏ đi, cuối cùng biến bài toán thành một bài toán hiển nhiên. Cho tới đây, tất cả phép toán ta thảo luận vẫn dựa trên tầng “lấy modulo”. Trong ví dụ cuối cùng, ta thậm chí tự xây dựng một kiểu modulo. Những ví dụ có thể áp dụng Euclid theo cách trực tiếp như vậy thật ra rất ít; thường thì ta chỉ vô tình suy ra một công thức làm nhỏ quy mô bài toán, nhưng chưa đạt tới mức có thể tính trực tiếp. Khi ấy ta có thể thử lặp lại quá trình này. Xem thêm các ví dụ ở phần 4.

3 Liên phân số và phương trình Pell

3.1 Liên phân số

Nhiều nội dung trong phần này tham khảo chương 4 về liên phân số đơn giản và phân số xấp xỉ tốt nhất trong bài tập đội tuyển năm 2006 của Dương Triết, *Nghiên cứu một lớp bài toán phân số*.

Một số thực ω có thể được biểu diễn bằng một dãy số nguyên dương $[a_0; a_1, a_2, \dots]$, trong đó

$$\omega = a_0 + \frac{1}{a_1 + \frac{1}{a_2 + \frac{1}{a_3 + \dots}}}$$

Cách biểu diễn này được gọi là biểu diễn **liên phân số**.

Hiển nhiên có thể tính $[a_0; a_1, a_2, \dots]$ ứng với ω theo cách sau:

$$\omega_0 \leftarrow \omega, \quad a_n = \lfloor \omega_n \rfloor, \quad \omega_{n+1} = \frac{1}{\omega_n - a_n}.$$

Quan sát quá trình này không khó thấy: nếu ω là số hữu tỉ, quá trình thực chất tương ứng với thuật toán Euclid. Vì vậy với mọi số hữu tỉ, dãy này luôn hữu hạn; còn với số vô tỉ, dãy chắc chắn vô hạn. Hơn nữa, nếu với dãy hữu hạn ta cấm số cuối cùng a_n bằng 1, thì dãy được xác định duy nhất.

Nếu cho một liên phân số hữu hạn bất kỳ $[a_0; a_1, a_2, \dots, a_n]$, hiển nhiên có thể tính lại phân số ban đầu bằng cách trực tiếp theo biểu thức. Ở đây chỉ xét một trường hợp đơn giản: khôi phục

$$a + \frac{1}{p/q} = \frac{ap + q}{p} = \frac{p'}{q'}.$$

Từ đây có vài nhận xét:

- Nếu p, q nguyên tố cùng nhau, thì p', q' cũng nguyên tố cùng nhau, nên quá trình này chỉ cần viết đơn giản thành phép nhân.
- Phương pháp này không thể dùng trực tiếp để tìm số hữu tỉ xấp xỉ một số vô tỉ đã cho, vì mỗi lần đều phải tính lại từ đầu.

- Quá trình này thực chất là một phép nhân ma trận:

$$\begin{pmatrix} p' \\ q' \end{pmatrix} = \begin{pmatrix} a & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} p \\ q \end{pmatrix}.$$

Từ điểm cuối có thể suy ra nhiều kết luận thú vị. Đặt

$$A_i = \begin{pmatrix} a_i & 1 \\ 1 & 0 \end{pmatrix}.$$

Khi đó có thể tính số hữu tỉ tương ứng với liên phân số bằng

$$A_0 A_1 \cdots A_n \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

Do phép nhân ma trận có tính kết hợp, ta hoàn toàn có thể tính trực tiếp từ trước ra sau. Như vậy vấn đề thứ hai ở trên đã được giải quyết về mặt toán học.

Thực tế, trên cơ sở này kết luận còn có thể được viết đẹp hơn. Đặt

$$S_i = A_0 A_1 \cdots A_i = \begin{pmatrix} x & y \\ z & w \end{pmatrix},$$

thì

$$S_{i+1} = S_i A_{i+1} = \begin{pmatrix} a_{i+1}x + y & x \\ a_{i+1}z + w & z \end{pmatrix}.$$

Dễ quan sát thấy tử số và mẫu số đều là các dãy truy hồi tuyến tính độc lập. Ma trận cuối cùng nhân thêm $\begin{pmatrix} 1 \\ 0 \end{pmatrix}$ chỉ có nghĩa là lấy $\frac{x_n}{y_n}$ làm phân số cần tìm; các chi tiết cũng trở nên hoàn chỉnh.

Cách tính đầy đủ là

$$\begin{aligned} p_n &= a_n p_{n-1} + p_{n-2}, & p_{-1} &= 1, & p_{-2} &= 0, \\ q_n &= a_n q_{n-1} + q_{n-2}, & q_{-1} &= 0, & q_{-2} &= 1. \end{aligned}$$

Sau đây là một số định nghĩa chuẩn hơn và vài kết luận không chứng minh; phần chứng minh có thể tham khảo bài của Dương Triết.

Định nghĩa 1. Nếu biểu diễn liên phân số của số thực ω là $[a_0; a_1, a_2, \dots]$, thì $[a_0; a_1, a_2, \dots, a_n]$ được gọi là phân số hội tụ thứ n của ω .

Định nghĩa 2. Một phân số p/q được gọi là phân số xấp xỉ tốt nhất của số ω khi và chỉ khi trong mọi phân số có mẫu số không vượt quá q , giá trị $|p/q - \omega|$ là nhỏ nhất.

Kết luận 4. Mọi phân số hội tụ của ω đều là phân số xấp xỉ tốt nhất. Kết luận này sẽ được dùng ở phần tiếp theo khi giới thiệu phương trình Pell.

3.2 Phương trình Pell

Nguồn bài. Bài toán kinh điển, SPOJ EQU2.

Đề bài. Với số nguyên dương N cho trước, trong đó N không phải số chính phương, hãy tìm nghiệm nguyên dương nhỏ nhất của phương trình

$$x^2 - Ny^2 = 1.$$

Phân tích và lời giải. Bài này cần giải phương trình Pell. Thực tế, mọi phương trình Diophantine bậc hai ẩn có dạng

$$ax^2 + by^2 + cxy + dx + ey + f = 0$$

đều có thể giải thông qua phương trình Pell.

Ở đây ta trực tiếp dẫn các kết luận do Lagrange và Euler phát hiện. Gọi p_k/q_k là các phân số hội tụ của $\sqrt{N}$.

Kết luận 5 (Lagrange). Tồn tại $n > 0$ sao cho

$$p_n^2 - Nq_n^2 = 1.$$

Kết luận 6 (Euler). Trong mọi n được nhắc tới ở kết luận 5, n nhỏ nhất biểu thị nghiệm nhỏ nhất (p_n, q_n) của phương trình Pell tương ứng.

Vì thế ta có thể tính dãy $\{(p_n, q_n)\}$ cho tới khi tìm được đáp án. Từ đó có thể viết ngay mã giả đầu tiên.

Algorithm 4 Naive Implementation of solving Pell's Equation

Input: a positive integer N , where N is not a perfect square number

Output: the minimum non-trivial solution of $x^2 - N*y^2 = 1$

```

1: omega[-1] <- sqrt(N)
2: p[-1] <- 1, p[-2] <- 0
3: q[-1] <- 0, q[-2] <- 1
4: for i = 0 to infinity do
5:   a[i] <- floor(omega[i-1])
6:   omega[i] <- 1 / (omega[i-1] - a[i])
7:   p[i] <- p[i-1] * a[i] + p[i-2]
8:   q[i] <- q[i-1] * a[i] + q[i-2]
9:   if p[i]^2 - N * q[i]^2 = 1 then
10:    return (p[i], q[i])
11:   end if
12: end for
```

Đáng tiếc, do sai số độ chính xác của số thực, thuật toán này cuối cùng có thể sai, thậm chí rơi vào vòng lặp vô hạn. Ta cần khai thác thêm thông tin.

Kết luận 7 (Euler, 1765). Tồn tại các số nguyên g_n và h_n sao cho

$$\omega_n = \frac{g_n + \sqrt{N}}{h_n}.$$

Nói cách khác, ở bất cứ thời điểm nào khi thuật toán chạy, ω đều có thể viết ở dạng trên, tức dạng vô tỉ bậc hai.

Thay $\omega_n = 1/(\omega_{n-1} - a_n)$ vào công thức trên:

$$\begin{aligned}
\omega_n &= \frac{1}{\omega_{n-1} - a_n} \\
&= \frac{1}{\frac{g_{n-1} + \sqrt{N}}{h_{n-1}} - a_n} \\
&= \frac{h_{n-1}}{\sqrt{N} + g_{n-1} - a_n h_{n-1}} \\
&= \frac{h_{n-1}(\sqrt{N} - g_{n-1} + a_n h_{n-1})}{(\sqrt{N} + g_{n-1} - a_n h_{n-1})(\sqrt{N} - g_{n-1} + a_n h_{n-1})} \\
&= \frac{\sqrt{N} - g_{n-1} + a_n h_{n-1}}{N - (-g_{n-1} + a_n h_{n-1})^2} \cdot \frac{h_{n-1}}{h_{n-1}}.
\end{aligned}$$

Từ đó suy ra quan hệ truy hồi của g_n và h_n :

$$\begin{aligned}
g_{-1} &= 0, & h_{-1} &= 1, \\
g_n &= -g_{n-1} + a_n h_{n-1}, & h_n &= \frac{N - g_n^2}{h_{n-1}}.
\end{aligned}$$

Tiến thêm một bước, thật ra cũng có thể tính a_n mà không để $\sqrt{N}$ tham gia trực tiếp:

$$\begin{aligned}
a_n &= \lfloor \omega_{n-1} \rfloor = \left\lfloor \frac{g_{n-1} + \sqrt{N}}{h_{n-1}} \right\rfloor \\
&= \left\lfloor \frac{g_{n-1} + \lfloor \sqrt{N} \rfloor}{h_{n-1}} \right\rfloor = \left\lfloor \frac{g_{n-1} + a_0}{h_{n-1}} \right\rfloor.
\end{aligned}$$

Sau phân tích trên, ta có thể viết một chương trình hoàn toàn không chứa sai số số thực trong quá trình tính toán, giải trọn vẹn bài toán.

Algorithm 5 Advanced Algorithm For Pell's Equation

Input: a positive integer N, where N is not a perfect square number

Output: the minimum non-trivial solution of $x^2 - N*y^2 = 1$

```

1: p[-1] <- 1, p[-2] <- 0
2: q[-1] <- 0, q[-2] <- 1
3: a[0] <- floor(sqrt(N)) // can be implemented without floats
4: g[-1] <- 0, h[-1] <- 1
5: for i = 0 to infinity do
6:   g[i] <- -g[i-1] + a[i] * h[i-1]
7:   h[i] <- (N - g[i]^2) / h[i-1]
8:   a[i+1] <- floor((g[i] + a[0]) / h[i])
9:   p[i] <- a[i] * p[i-1] + p[i-2]
10:  q[i] <- a[i] * q[i-1] + q[i-2]
11:  if p[i]^2 - N * q[i]^2 = 1 then
12:    return (p[i], q[i])
13:  end if
14: end for

```

4 Một vài ứng dụng khác

4.1 Phân số có mẫu nhỏ nhất giữa hai phân số

Nguồn bài. Chương 3 trong bài tập của Dương Triết.

Đề bài. Tìm phân số p/q có mẫu số nhỏ nhất thỏa mãn

$$\frac{a}{b} < \frac{p}{q} < \frac{c}{d}.$$

Phân tích và lời giải. Lời giải trong bài của Dương Triết đã rất đầy đủ; ở đây chỉ trình bày tóm lược. Trước hết, nếu tồn tại số nguyên n thỏa mãn

$$\frac{a}{b} < n < \frac{c}{d},$$

thì bài toán được giải trực tiếp. Nếu không, có thể chuyển bài toán thành tìm p/q thỏa mãn

$$0 \leq \frac{a \bmod b}{b} < \frac{p}{q} < \frac{c \bmod d}{d} < 1.$$

Có thể chứng minh rằng q thỏa mãn điều kiện là duy nhất, và khi ấy p cũng nhỏ nhất. Vì vậy có thể lấy nghịch đảo để chuyển bài toán thành

$$\frac{d}{c \bmod d} < \frac{q}{p} < \frac{b}{a \bmod b}.$$

Công thức trên hiển nhiên chính là chia Euclid. Bằng cách lấy nghịch đảo mà bài toán không đổi, ta bắt ngờ bước vào giai đoạn lấy modulo, từ đó xây dựng được thuật toán.

4.2 Số điểm lưới trong đa giác đơn có đỉnh hữu tỉ

Nguồn bài. TopCoder SRM 410 WifiPlanet, tác giả bài là bmerly; bài được trích dẫn chỉ nhằm mục đích giáo dục.

Đề bài. Trên mặt phẳng điểm lưới nguyên có một đa giác đơn, tọa độ các đỉnh đều là số hữu tỉ. Hỏi bên trong nó có bao nhiêu điểm lưới. Đề bài đảm bảo không có điểm lưới nào nằm trên biên.

Phân tích và lời giải. Đây là bài cuối của vòng thi đó, và trong lúc thi không có lời giải đúng nào.

Liên hệ với cách tính diện tích đa giác thông thường, dễ nghĩ rằng bài này cần phân hoạch đa giác. Nhưng phép tam giác phân thông thường không phù hợp ở đây, ít nhất lời giải sẽ không đơn giản, vì đếm số điểm lưới bên trong một tam giác thuần túy là việc rất phức tạp. Vì vậy nghĩ tới phân hoạch hình thang là khá tự nhiên.

Hình thức hóa hơn, ta cần tính có bao nhiêu điểm lưới nằm dưới một đoạn thẳng. Nhờ giả thiết của đề, trên cạnh không có điểm lưới, nên bài toán đơn giản hơn. Cụ thể, cho $(x_1, y_1), (x_2, y_2)$ với $x_1 < x_2$, hỏi số điểm lưới (x, y) thỏa mãn

$$x_1 \leq x < x_2, \quad 0 \leq y < \frac{y_1(x_2 - x) + y_2(x - x_1)}{x_2 - x_1}.$$

Vì x, y đều là số nguyên, bài toán có thể chuyển thành đếm số nút dưới đoạn thẳng nối (x_1, y_1) và (x_2, y_2) , trong đó $x_1, x_2 \in \mathbb{Z}$ còn $y_1, y_2 \in \mathbb{Q}$.

Thực tế, việc tính số này tương đương với tính giá trị

$$\sum_{i=0}^{n-1} \left\lfloor \frac{a + di}{m} \right\rfloor.$$

Đến đây có hai cách giải:

- Cách thứ nhất dễ nghĩ hơn, nhưng độ phức tạp là $\mathcal{O}(\sqrt{n})$ và cần xét quá nhiều chi tiết.
- Cách còn lại là lời giải chính thức, một biến thể của thuật toán Euclid, có độ phức tạp $\mathcal{O}(\log n)$.

Trước hết giới thiệu thuật toán $\mathcal{O}(\sqrt{n})$. Đặt

$$A_i = \left\lfloor \frac{a + di}{m} \right\rfloor.$$

Xét $A_i - A_{i-1}$, có hai khả năng:

- Nếu $(a + d(i - 1)) \bmod m + d \bmod m \geq m$, thì

$$A_i - A_{i-1} = 1 + \left\lfloor \frac{d}{m} \right\rfloor.$$

- Nếu $(a + d(i - 1)) \bmod m + d \bmod m < m$, thì

$$A_i - A_{i-1} = \left\lfloor \frac{d}{m} \right\rfloor.$$

Muốn tính nhanh, tốt nhất ta có thể giảm $d \bmod m$ càng nhiều càng tốt, để sau đó tính một lần khoảng m/d giá trị. Một cách là chọn một số c , rồi phân loại n số $0, \dots, n - 1$ theo giá trị modulo c ; mỗi lớp như vậy có thể cộng trực tiếp $dc \bmod m$. Cụ thể, đặt $c \in [1, C]$ sao cho $dc \bmod m$ nhỏ nhất. Giả sử $dk \bmod m$ phân bố đều khi $k \in [1, n]$, thì kỳ vọng $dc \bmod m < m/C$. Vì thế với mọi $i \in [0, c)$, ta trực tiếp liệt kê tổng A_{i+jc} . Quá trình này có độ phức tạp thực tế là $\mathcal{O}(C + n/C)$; khi $C = \sqrt{n}$, độ phức tạp nhỏ nhất là $\mathcal{O}(\sqrt{n})$.

Algorithm 6 $\mathcal{O}(\sqrt{n})$ implementation

Input: a, d, m, n

Output: $\sum_{i=0}^{n-1} \text{floor}((a + d*i) / m)$

```

1: C <- sqrt(n), c <- 1
2: for i = 1 to C do
3:   if d*i mod m < d*c mod m then
4:     c <- i
5:   end if
6: end for
7: for i = 0 to c - 1 do
8:   j <- i
9:   while j < n do
10:    let t be the maximum number such that
      (d*j mod m) + (d*c mod m) * t < m
11:    apply changes, add the answer to ret
12:    j <- j + t
13:  end while
14: end for
15: return ret

```

Mã giả trên cho thấy thuật toán vẫn có rất nhiều chi tiết, nhiều chỗ phải xử lý cẩn thận mới giải được bài. Trong đề này n, m được đặt tới 10^9 nên thuật toán này chỉ vừa đủ qua kiểm thử. Nếu quy mô n lớn hơn, ta sẽ bất lực; vì vậy cần tìm một thuật toán hiệu quả, ngắn gọn và đẹp hơn.

Xét biểu thức

$$\sum_{i=0}^{n-1} \left\lfloor \frac{a + di}{m} \right\rfloor. \quad (1)$$

Ta có thể đặt vài quy ước:

- $0 \leq a < m$; nếu không, có thể trực tiếp đưa $[a/m]$ ra ngoài. Trực giác hình học là phép biến đổi này biến một “hình thang” thành một “tam giác”.
- $0 < d < m$; bằng phương pháp giống trên có thể đảm bảo điều này. Chú ý rằng nếu $d = 0$, dãy trở thành dãy hằng và có thể tính trực tiếp. Về bản chất hình học, phép biến đổi này có thể hiểu là: nếu cạnh góc vuông kia dài hơn, ta dùng biến đổi này để khiến cạnh góc vuông hiện tại trở thành cạnh dài nhất.

Nhận xét thứ hai gợi ý rằng chỉ cần hoán đổi được hai “cạnh góc vuông” là ta có thể thuận lợi áp dụng thuật toán Euclid.

Nhìn biểu thức (1) từ góc độ khác: có một dãy điểm $(i, a + di)$, và từ mỗi điểm kẻ một tia thẳng xuống dưới. Đặt

$$l = \left\lfloor \frac{a + dn}{m} \right\rfloor.$$

Khi đó có l đường thẳng ngang $y = km$. Ý nghĩa của (1) chính là tổng số giao điểm của mọi tia với mọi đường thẳng ngang ấy.

Đổi góc nhìn lần nữa, số giao điểm cũng có thể hiểu là tổng số điểm nằm phía trên mỗi đường thẳng ngang, tức

$$\left\lfloor \frac{topY - nowY}{d} \right\rfloor,$$

trong đó $topY = a + dn$, còn $nowY$ là độ cao của đường thẳng ngang hiện tại.

Mô tả hình. Hình cuối trong bài gốc vẽ một bó tia đi xuống từ các điểm trên đường thẳng, cắt các đường ngang $y = km$. Nó minh họa việc chuyển từ “đếm giao điểm trên các tia dọc” sang “đếm số điểm phía trên từng đường ngang”, với hai mức độ cao được ghi là $topY$ và $nowY$.

Từ đây có thể viết ngay công thức

$$\sum_{i=0}^{n-1} \left\lfloor \frac{a + di}{m} \right\rfloor = \sum_{k=0}^{l-1} \left\lfloor \frac{(a + dn) \bmod m + mk}{d} \right\rfloor. \quad (2)$$

Theo (2), ta đã có thể viết một chương trình hoàn chỉnh. Quay lại phân tích trực giác ở trên, sau khi có công thức này, ta có thể liên tục lấy modulo hai “cạnh góc vuông” rồi hoán đổi chúng. Hiển nhiên độ phức tạp là $\mathcal{O}(\log n)$. Tới đây bài toán được giải quyết.

5 Kết luận

Dù Euclid đã rất quen thuộc với đa số thí sinh thi tin học, trong ứng dụng thực tế nó lại khó được nghĩ tới. Thật ra, chỉ cần thoát khỏi khuôn mẫu tư duy và đơn giản hóa bài toán từ một số đẳng thức tưởng như không liên quan, cuối cùng ta đều có thể đạt tới mục tiêu. Bài viết này chủ yếu giới thiệu một vài ứng dụng của thuật toán Euclid từ góc độ tư tưởng, xem như một cách mở rộng tầm nhìn. Cuối cùng có thể thấy mọi bài toán đều là bài toán toán học thuần túy, điều này cũng phần nào thể hiện vai trò của tư tưởng toán học trong thi tin học.

Tài liệu tham khảo

1. Wikipedia, <http://www.wikipedia.org/>.
2. MathWorld, <http://mathworld.wolfram.com/>.
3. Dương Triết, *Nghiên cứu một lớp bài toán phân số*.

Lời cảm ơn

Xin cảm ơn thầy Tào Văn của Trường Trung học Thường Châu; không có thầy thì không có tôi của hiện tại. Xin cảm ơn Vladislav Isenbaev (WiNGeR) của SPb IFMO đã cung cấp đề bài, và cảm ơn sự giúp đỡ của Ngô Trác Kiệt từ Đại học Giao thông Thượng Hải cùng Chu Hoằng Thừa từ Đại học Khoa học và Công nghệ Trung Quốc.